Artificial Intelligence Search Algorithms

A Comprehensive Overview & Paraphrased Analysis

---

1. Introduction to AI Search Methods

Search algorithms in Artificial Intelligence are foundational tools utilized across various domains, including robotics, optimization tasks, and strategic game playing. In adversarial environments like Chess or Tic-Tac-Toe, these algorithms enable an intelligent agent to anticipate future states and determine the optimal move. This document reviews four primary search techniques: Minimax, Alpha-Beta Pruning, Heuristic Alpha-Beta Search, and Monte-Carlo Tree Search (MCTS).

2. The Minimax Algorithm

Minimax is a fundamental recursive strategy for decision-making in two-player zero-sum games. It operates on a simple premise: one player (MAX) aims to maximize their score, while the opponent (MIN) seeks to minimize it. By exhaustively exploring the entire game tree, Minimax guarantees the mathematically optimal move. However, its primary drawback is its inefficiency; evaluating every possible node becomes computationally unfeasible for games with vast search spaces.

3. Alpha-Beta Pruning

To overcome the performance bottlenecks of the standard Minimax algorithm, Alpha-Beta Pruning introduces an optimization technique that dramatically reduces the number of nodes evaluated. It works by maintaining two values: **Alpha** (the best minimum score the MAX player can secure) and **Beta** (the best maximum score the MIN player can secure). If at any point Alpha becomes greater than or equal to Beta, the algorithm "prunes" or discards that branch, knowing it will not affect the final decision. This yields the exact same optimal result as Minimax but in a fraction of the time.

4. Heuristic Alpha-Beta Search

For particularly massive game trees where even Alpha-Beta pruning takes too long, Heuristic Alpha-Beta Search is employed. This variant restricts the search to a specific depth rather than exploring until the game ends. Because it stops mid-game, it uses a *heuristic evaluation function* to estimate the quality of the board state—for instance, rewarding positions that threaten the opponent or are close to a win. While it only provides an approximate optimal move, it is highly practical and scalable for real-world AI applications.

5. Monte-Carlo Tree Search (MCTS)

MCTS is a probabilistic algorithm famous for its success in modern AI milestones like AlphaGo. Instead of exhaustive evaluation, it relies on random simulations to gauge the best possible action. The algorithm executes four iterative phases:

- **Selection:** Navigating the tree to find the most promising child node using the Upper Confidence Bound (UCB) formula.
- **Expansion:** Adding one or more child nodes to grow the tree.
- **Simulation (Playout):** Running a randomized game from the new node until a terminal state is reached.
- **Backpropagation:** Updating the path back to the root with the results of the simulation.

MCTS excels in immensely complex games because it focuses its computational power on the most promising branches without requiring a domain-specific heuristic function.

6. Algorithm Comparison

Algorithm	Provides Optimal Move?	Uses Pruning?	Execution Speed	Viability for Large Games
Minimax	Yes	No	Slow	Poor
Alpha-Beta Pruning	Yes	Yes	Moderate	Moderate
Heuristic Alpha-Beta	Approximate	Yes	Fast	High
MCTS	Probabilistic	No (Uses statistical sampling)	Very Fast	Excellent